

Intelligent Phishing URL Detection

1. Abstract

Phishing attacks are one of the most common cybersecurity threats used to steal sensitive user information through fake websites and malicious URLs. Traditional phishing detection methods often struggle to detect newly generated phishing websites, creating the need for more intelligent detection systems.

This project presents a phishing URL detection system using machine learning and deep learning techniques. The dataset was preprocessed through data cleaning, duplicate removal, balancing using undersampling, and normalization using Min-Max Scaling. Three models were implemented and evaluated: Logistic Regression, Random Forest, and Artificial Neural Network (ANN).

The models were evaluated using Accuracy, Precision, Recall, and F1-Score.

Experimental results showed that the Random Forest model achieved the best overall and most stable performance with 94.36% accuracy, outperforming Logistic Regression and demonstrating slightly better consistency than the ANN model in phishing URL detection. The ANN model also achieved strong performance, reaching an average accuracy of 93.22% across multiple runs after hyperparameter tuning.

2. Introduction and problem statement(s)

Phishing attacks are considered one of the most common and dangerous cybersecurity threats. Attackers use fake or malicious websites to trick users into revealing sensitive information such as usernames, passwords, banking details, and personal data. As phishing techniques continue to evolve, traditional rule-based detection systems become less effective in identifying newly generated phishing URLs. Therefore, machine learning and deep learning techniques have become important solutions for improving phishing detection systems.

This project focuses on detecting phishing URLs using machine learning and deep learning models. The main objective is to build an intelligent classification system capable of distinguishing between legitimate and phishing URLs based on extracted URL features. The project uses a phishing URL dataset containing multiple features related to URL structure, domain characteristics, HTTPS information, and other security indicators.

Before model training, several preprocessing steps were applied to prepare the dataset. These steps included correcting feature names, removing unnecessary columns, handling duplicate records, encoding the target labels, balancing the training data using undersampling, and normalizing the feature values using Min-Max Scaling. These preprocessing techniques were necessary to improve data quality and ensure better model performance. The input of the system consists of numerical URL-based features, while the output is a binary classification result: 0 = Legitimate URL and 1 = Phishing URL.

To solve the problem, multiple models were implemented and compared. Logistic Regression was used as the baseline machine learning model, random Forest was implemented as an advanced machine learning model due to its strong classification capabilities and stability, and an Artificial Neural Network (ANN) was used as the deep learning model.

The models were evaluated using several performance metrics, including Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and Classification Report. These evaluation methods were used to analyze model performance and determine the most effective approach for phishing URL detection.

Experimental results showed that the Random Forest model achieved the best overall and most stable performance among the evaluated models.

3. Literature Review

Many previous studies have focused on improving phishing URL detection using machine learning and deep learning techniques.

Sahoo et al. (2017) discussed different machine learning approaches used for malicious URL detection. The study explained how URL-based features can help classify websites as legitimate or malicious. Their work showed that machine learning models can improve phishing detection accuracy compared to traditional detection methods. This is similar to our project since we also used URL features to classify phishing and legitimate URLs.

Another important study by **Breiman (2001)** introduced the Random Forest algorithm, which combines multiple decision trees to improve classification performance and reduce overfitting. Random Forest proved to be effective in handling complex data patterns and achieving stable prediction results. In our project, Random Forest was also implemented as an improved machine learning model and achieved strong performance in phishing URL detection.

In addition, **Aljofey et al. (2020)** proposed a phishing detection approach using URL and HTML features with advanced machine learning and deep learning techniques. Their study showed that deep learning models can learn complex patterns within phishing URLs and improve overall detection performance. This study is closely related to our work because we also implemented an ANN deep learning model and performed several experiments to improve its performance through hyperparameter tuning.

Based on these previous studies, our project focused on comparing different machine learning and deep learning models for phishing URL detection. We also performed multiple ANN optimization experiments to improve deep learning performance. However, the final comparison showed that the Random Forest model achieved the best overall performance and stability in this project.

4. Methods/ Model

Data Preprocessing

Before training the machine learning and deep learning models, several preprocessing steps were applied to prepare the dataset, improve data quality, and make it suitable for phishing URL classification.

1. Feature Name Correction

The feature names were corrected to make the dataset easier to understand and process. Clear and consistent feature names help during data analysis, model training, and interpretation of results.

2. Removing Unnecessary Columns

Unnecessary columns were removed because they did not contribute to phishing URL classification. Removing irrelevant columns helps reduce noise in the dataset and allows the models to focus only on meaningful URL-based features.

3. Removing Duplicate Records

Duplicate records were removed to prevent the models from learning repeated patterns. This step helps ensure fair training and avoids giving repeated samples more influence during the learning process.

4. Target Variable Encoding

The target variable was encoded into numerical values to make it suitable for machine learning models. In this project, 0 represents a legitimate URL, while 1 represents a phishing URL.

5. Train-Test Split

The dataset was split into training and testing sets using an 80:20 ratio. The parameter `test_size=0.2` means that 20% of the dataset was used for testing, while 80% was used for training.

The parameter `random_state=0` was used to ensure reproducibility, meaning that the same split can be obtained every time the code is executed. In addition, `stratify=y` was used to preserve the same class distribution in both the training and testing sets.

6. Handling Class Imbalance Using Undersampling

To handle class imbalance, undersampling was applied to the training data only. This step was important to reduce model bias toward the majority class. By balancing the training data, the models were able to learn from both legitimate and phishing URL samples more fairly.

7. Feature Scaling Using Min-Max Scaling

Min-Max Scaling was applied to normalize the feature values into a similar range. This step is especially important for models such as Logistic Regression and ANN because they are sensitive to differences in feature scales.

By scaling the features, the models can train more effectively and avoid giving larger numerical values more influence than smaller values.

Logistic Regression Model Methodology :

A Logistic Regression model was implemented as the baseline machine learning model for phishing URL classification. The model was selected because it is simple, efficient, and widely used for binary classification tasks. Since the dataset contains two classes (phishing and legitimate URLs), Logistic Regression was considered suitable as an initial benchmark model.

Before training the model, the dataset underwent preprocessing steps including data balancing using undersampling, feature scaling using `MinMaxScaler`, and splitting the dataset into training and testing sets. In addition, the target variables (`y_train` and `y_test`) were converted from object type to integer type to ensure compatibility with the model.

The model was created using: `LogisticRegression(max_iter=1000, random_state=42)`

where `max_iter=1000` was used to allow sufficient iterations for convergence, and `random_state=42` ensured reproducibility of the results.

The model was then trained using the training dataset and evaluated on the testing dataset. Performance evaluation was conducted using several metrics including Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and Classification Report.

Random Forest Model Methodology:

Random Forest model was implemented as an improved machine learning model for phishing URL classification. Random Forest was selected because it builds multiple decision trees and combines their predictions, allowing the model to capture more complex and nonlinear relationships between URL features compared to Logistic Regression.

The Random Forest model was configured using the following hyperparameters:

- 1. `n_estimators = 101`:** The model builds 101 decision trees to improve prediction stability, reduce variance, and avoid tie voting in the majority vote process.
- 2. `max_depth = 10`:** The maximum depth of each tree was limited to 10 to help reduce overfitting.
- 3. `random_state = 42`:** Ensures reproducibility of results across different runs.

The model was trained using the preprocessed training dataset and evaluated on the testing dataset. Performance evaluation was conducted using Accuracy, Precision, Recall, F1-Score, AUC Score, Confusion Matrix, and Classification Report to measure the model's ability to correctly distinguish phishing URLs from legitimate URLs.

In addition, 5-fold Cross Validation was applied to evaluate model generalization and ensure stable performance across different data splits. Feature Importance Analysis was also performed to identify the most influential URL features contributing to phishing detection.

Artificial Neural Network Model Methodology:

Artificial Neural Network (ANN) was implemented as an advanced deep learning approach for phishing URL classification due to its ability to learn complex and nonlinear relationships between URL features.

Before training, the dataset was preprocessed using `MinMaxScaler`, and the data was divided into training and testing sets. The ANN model was developed using multiple hidden layers with ReLU activation functions and a Sigmoid output layer for binary classification.

Several ANN experiments were conducted by modifying hyperparameters such as the number of hidden layers, neurons, dropout rates, and training epochs to improve model performance and reduce overfitting. During tuning, the ANN achieved up to 95% accuracy in some runs. However, slight variations in performance were observed across repeated executions due to the stochastic nature of neural network training, including random weight initialization and dropout behavior.

To provide a more reliable evaluation, the final ANN configuration was executed multiple times under the same settings, and the average performance was calculated instead of relying on a single execution result .

The obtained ANN execution accuracies were:

- Run 1 = 95%
- Run 2 = 94%
- Run 3 = 93%

The calculated average ANN performance achieved:

- Average Accuracy = 94%
- Average Precision = 94.67%
- Average Recall = 94%
- Average F1-Score = 94%

Although the ANN model demonstrated strong phishing URL detection capability, the Random Forest model maintained slightly better stability and reproducibility across executions. In addition, Random Forest achieved the highest stable Recall (95.23%), making it the most reliable final model for deployment in this project.

5. Results

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	92.05%	91.71%	91.87%	91.79%
Random Forest	94.36%	93.25%	95.23%	94.23%
ANN Experiment 1 (Initial Model)	92.74%	95.12%	89.58%	92.27%
ANN Experiment 2 (Tuned Model)	93.76%	93.32%	93.82%	93.57%

ANN Experiment 3 (Final Tuned Model (Average))	93.22%	95.68%	90.16%	92.78%
--	--------	--------	--------	--------

The experimental results demonstrated that all implemented machine learning and deep learning models achieved good performance in phishing URL detection. Several evaluation metrics were used to measure model performance, including Accuracy, Precision, Recall, and F1-Score.

Logistic Regression was used as the baseline machine learning model and achieved 92.05% Accuracy, 91.71% Precision, 91.87% Recall, and 91.79% F1-Score.

Random Forest achieved the best overall and most stable performance among all evaluated models, with 94.36% Accuracy, 93.25% Precision, 95.23% Recall, and 94.23% F1-Score. The model demonstrated strong phishing URL detection capability and achieved the highest Recall performance, making it highly effective in minimizing false negatives. The Random Forest model also achieved an AUC Score of 0.99, indicating excellent capability in distinguishing phishing URLs from legitimate URLs.

Three ANN experiments were conducted to improve deep learning performance through hyperparameter tuning and architecture optimization. ANN Experiment 1 achieved 92.74% Accuracy, 95.12% Precision, 89.58% Recall, and 92.27% F1-Score. ANN Experiment 2 improved the model performance and achieved 93.76% Accuracy, 93.32% Precision, 93.82% Recall, and 93.57% F1-Score.

For the final ANN configuration, multiple executions were performed under the same settings due to the stochastic behavior of neural network training. The final ANN Experiment 3 results were therefore reported using the average performance across repeated runs. The final averaged ANN model achieved 93.22% Accuracy, 95.68% Precision, 90.16% Recall, and 92.78% F1-Score.

Based on the final comparison results, Random Forest was selected as the final proposed model due to its superior stability, strong overall performance, and highest Recall performance in phishing URL detection, while the ANN model demonstrated competitive deep learning performance after hyperparameter tuning.

Confusion Matrix (Random Forest):

Actual / Predicted	Predicted Legitimate	Predicted Phishing
Actual Legitimate	565	39
Actual Phishing	27	539

6. Discussion and Analysis

The results showed a clear performance progression across all implemented models. Logistic Regression achieved 92.05% accuracy, but its linear decision boundary limited its ability to capture complex URL feature relationships, resulting in a higher number of false negatives compared to the other models.

Random Forest improved performance to 94.36% accuracy and achieved the highest Recall performance at 95.23%, which is especially important in cybersecurity because fewer phishing URLs escaped detection. The model also achieved an AUC Score of 0.99, indicating excellent capability in distinguishing phishing URLs from legitimate URLs. The 5-fold Cross Validation confirmed stable generalization, and Feature Importance Analysis showed that SSL-related and URL-based features were the strongest predictors of phishing activity.

For the ANN experiments, three architectures were tested. Experiment 1 used a shallow 2-layer network and achieved 92.74% Accuracy, 95.12% Precision, 89.58% Recall, and 92.27% F1-Score. Experiment 2 introduced Batch Normalization and Dropout, improving the model performance to 93.76% Accuracy, 93.32% Precision, 93.82% Recall, and 93.57% F1-Score. Experiment 3 applied further hyperparameter tuning using a wider architecture, smaller batch size, lower learning rate, and additional training epochs. Due to the stochastic nature of neural network training, Experiment 3 was executed multiple times and the final results were reported using the average performance across runs. The final averaged ANN Experiment 3 achieved 93.22% Accuracy, 95.68% Precision, 90.16% Recall, and 92.78% F1-Score.

Based on the final comparison, Random Forest was the most suitable model for this project because it achieved the best overall balance across Accuracy, Recall, stability, and reproducibility. This is especially important in cybersecurity because high Recall helps reduce false negatives, which represent phishing URLs incorrectly classified as legitimate.

Limitations

Several limitations were identified in this project. First, undersampling reduced the training data size, which may have limited the models' exposure to diverse phishing patterns. Second, the ANN results showed variability across runs due to random weight initialization, making results slightly inconsistent between executions. Third, the models were trained and tested on a static dataset and were not evaluated on real-world live URLs, which may affect generalization to newly generated phishing websites.

Future Improvements

To address these limitations, future work could apply SMOTE oversampling to preserve more training samples instead of discarding majority class data. Additionally, automated hyperparameter tuning using Keras Tuner could produce more stable and optimized ANN configurations. Finally, testing the model on live URL data would provide a more realistic evaluation of its detection capability against evolving phishing threats.

7. References

- Sahoo, D., Liu, C., & Hoi, S. C. H. (2017). *Malicious URL detection using machine learning: A survey*. arXiv. <https://arxiv.org/abs/1701.07179>
- Cox, D. R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society. Series B (Methodological)*, 20(2), 215–242. <https://doi.org/10.1111/j.2517-6161.1958.tb00292.x>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Aljofey, A., Jiang, Q., Rasool, A., Chen, H., & Liu, W. (2020). An effective detection approach for phishing websites using URL and HTML features. *Scientific Reports*, 10(1), 8842. <https://doi.org/10.1038/s41598-020-64745-3>
- Scikit-learn Developers. (2026). Preprocessing data. scikit-learn Documentation. <https://scikit-learn.org/stable/modules/preprocessing.html>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.